MODELO CONCEPTUAL DEL DOMINIO

Es un modelo de objetos que se centra en mostrar un "panorama general" del mismo en el que se incluyen las principales **clases**, sus **atributos** y sus **relaciones**.

El modelo del dominio debe poder extraerse de los requisitos. Debe describir objetos del mundo real, NO artefactos software.

¿Cómo encontrar clases conceptuales?

Identificando nombres y frases nominales: Los nombres en las descripciones textuales del dominio (casos de uso, etc.) indican candidatos a clases conceptuales y/o atributos.

Asociación conceptual

Relación semántica entre dos o más clasificadores que involucran conexiones entre sus instancias.

Una asociación conceptual NO ES: Un flujo de datos, una relación de clave foránea, un atributo de instancia, una conexión entre objetos software.

Una asociación conceptual ES: Una relación entre dos objetos desde una perspectiva conceptual (es decir, en el mundo real).

Eso no evita que muchas asociaciones conceptuales se conviertan en asociaciones de un modelo de diseño.

Atributos en modelos del dominio

Los atributos deben ser preferentemente tipos básicos de datos (boolean, number, character) o tipos no tan básicos, pero de uso cotidiano (date, time, address, phone, etc.). Generalmente su tipo concreto se omite ya que no es relevante a nivel conceptual.

Los atributos nunca deberán identificar a otra clase del dominio, la relación entre clases del dominio deberá hacerse a través de asociaciones.

MODELOS DEL DISEÑO

- **Modelo estático**: Representa la estructura del programa en base a paquetes y objetos que se relacionan.
- **Modelo dinámico**: Representa el comportamiento del programa en base a objetos que se comunican entre sí.

RESPONSABILIDAD DE CREACIÓN

Una clase B tiene la responsabilidad de crear una instancia de la clase A si se cumple que:

- B "contiene" o "agrega" objetos de tipo A.
- B registra instancias de objetos de A.
- B utiliza estrechamente objetos de A.
- B tiene los datos de inicialización de A.

Cuantas más condiciones cumpla mejor. En caso de que varias clases cumplan las condiciones, la que cumpla la primera condición tiene prioridad.

@informaticapau

ANOTACIONES VS. INTERFACES DE MARCADO

Decimos que los interfaces de marcado no son un uso adecuado de los interfaces porque, a partir de la versión 5 de Java existe un mecanismo más útil para realizar ese marcado: las anotaciones.

Anotaciones

Son metadatos, es decir, información sobre los elementos del programa.

Son similares a las etiquetas del javadoc (como @author) pero aplicadas al código, no a los comentarios.

Existen distintas anotaciones creadas en el API de Java (@Override, @Deprecated, @SupressWarnings, etc.) pero su verdadera potencia proviene de la posibilidad de crear anotaciones personalizadas.

Al contrario que con los interfaces, con las anotaciones se pueden anotar no sólo clases sino también métodos o atributos.

La anotación @Deprecated

Declara un método como depreciado para que el compilador avise de que no debe usarse.

Es diferente a la misma anotación usada en Javadoc y que, sorprendentemente, el compilador también leía (y lo sigue haciendo por motivos de compatibilidad hacia atrás).

Actualmente se recomienda usar las dos anotaciones, una para los comentarios y otra para el compilador.

IMPORTACIÓN ESTÁTICA

Es similar a la importación de clases, pero en este caso se importan elementos estáticos de las clases (atributos y métodos). Se pueden usar los elementos estáticos sin anteponer el nombre de la clase. Los elementos importados aparentan ser de la propia clase cuando en realidad son de otra, produce confusión. En caso de usarla optar preferentemente por la importación de elementos concretos evitando usar la notación "*".

ABSTRACCIÓN: Representación de las características fundamentales de algo sin incluir antecedentes o detalles irrelevantes.

AUTOBOXING: Proceso mediante el cual un tipo básico es convertido automáticamente en su clase envoltorio cuando se requiere que se comporte como un objeto.

AUTOUNBOXING: Proceso mediante el cual un objeto de una clase envoltorio es convertido automáticamente en su tipo básico cuando se requiere que se comporte como dicho tipo básico.

BOXING: Proceso de introducir un tipo básico en su correspondiente clase envoltorio.

CLASE: Plantilla que describe la estructura y el comportamiento de un tipo de objetos y permite la creación de los mismos.

CLASE ABSTRACTA: Clase que no pueden ser instanciada (no se puede crear un objeto de la misma) y que está destinada a ser extendida por herencia.

@informaticapau

CLASE ENVOLTORIO (*WRAPPER CLASSES*): Clase que envuelve a un tipo básico y le proveen del interfaz de objeto que le hace falta para poder ir en una colección de objetos.

CLASE GENÉRICA: Clase que incluye un parámetro de tipo (T) que representa a un objeto cualquiera cuyo tipo real será especificado a la hora de crear instancias de dicha clase.

COACCIÓN (COERCIÓN): Operación semántica por la cual se convierte un argumento al tipo esperado por una función para evitar que se produzca un error de tipos.

COHESIÓN: Es a la medida en que las funciones que incluye un objeto están relacionadas funcionalmente entre sí.

COMODÍN (*WILDCARD*): Tipo especial de parámetro que representa a un tipo desconocido en la declaración de tipos genéricos.

COMPOSICIÓN: Define relaciones "TIENE UN" y ocurre cuando un elemento contiene otros elementos (los objetos incluyen otros objetos en la definición de su estado).

COVARIANZA: Propiedad de los subtipos de seguir actuando como tales en tipos más complejos.

ENCAPSULAMIENTO: Proceso de ocultación al exterior de aquellos detalles de la implementación de un objeto que no es necesario conocer para su uso.

HASH: Es una función que mapea datos de un tamaño arbitrario en datos de un tamaño fijo.

HERENCIA: Relación "ES UN" entre clases, en las que una clase comparte la estructura de comportamiento definidos en una o más clases.

IDENTIDAD: Dos objetos son idénticos si y solo si son el mismo objeto (es decir, tienen un mismo OID).

IGUALDAD: Dos objetos se consideran iguales si, a pesar de ser objetos distintos (con distinto OID), sus atributos son idénticos.

INTERFACES: Son clases abstractas "puras" que definen un protocolo de conducta, pero no cómo debe implementarse dicha conducta.

LIGADURA: Proceso que se encarga de ligar o relacionar la llamada a un método (o mensaje) con el código que se ejecuta finalmente.

MARCADOR DE CLASE: Interfaz que se define sin incluir ningún método ni atributo

MENSAJE: Petición enviada a un objeto para que realice una acción y que va acompañada de información adicional (argumentos) necesaria para la realización de la misma.

MÉTODO ABSTRACTO: Método en el que sólo se define su interfaz (nombre, parámetros y tipo de retorno) pero no su implementación (que se difiere a las subclases).

MODELO: Simplificación de la realidad que permite comprender mejor el sistema que estamos desarrollando.

MODULARIDAD: Propiedad que tiene un sistema que ha sido descompuesto en un conjunto de partes o módulos que sean cohesivos (guardan cierta relación lógica) y débilmente acoplados (minimizan sus dependencias).

@informaticapau

MÓDULOS: Agrupan procedimientos con funcionalidades comunes. Incluyen procedimientos y datos privados al módulo.

MUTABLE: Que puede modificarse después de haber sido creado.

OBJETO: Elemento identificable que contiene características declarativas (que determinan su estado) y características procedimentales (que modelan su comportamiento).

POLIMORFISMO: Capacidad de un objeto de pertenecer a más de una clase y de una función de ser aplicada sobre parámetros de distintas clases.

PROCEDIMIENTOS: Conjunto de instrucciones y datos internos identificados por un nombre.

SOBRECARGA (*OVERLOADING*): Consiste en utilizar el mismo nombre para denotar métodos distintos diferenciándolos por el numero o tipo de los parámetros.

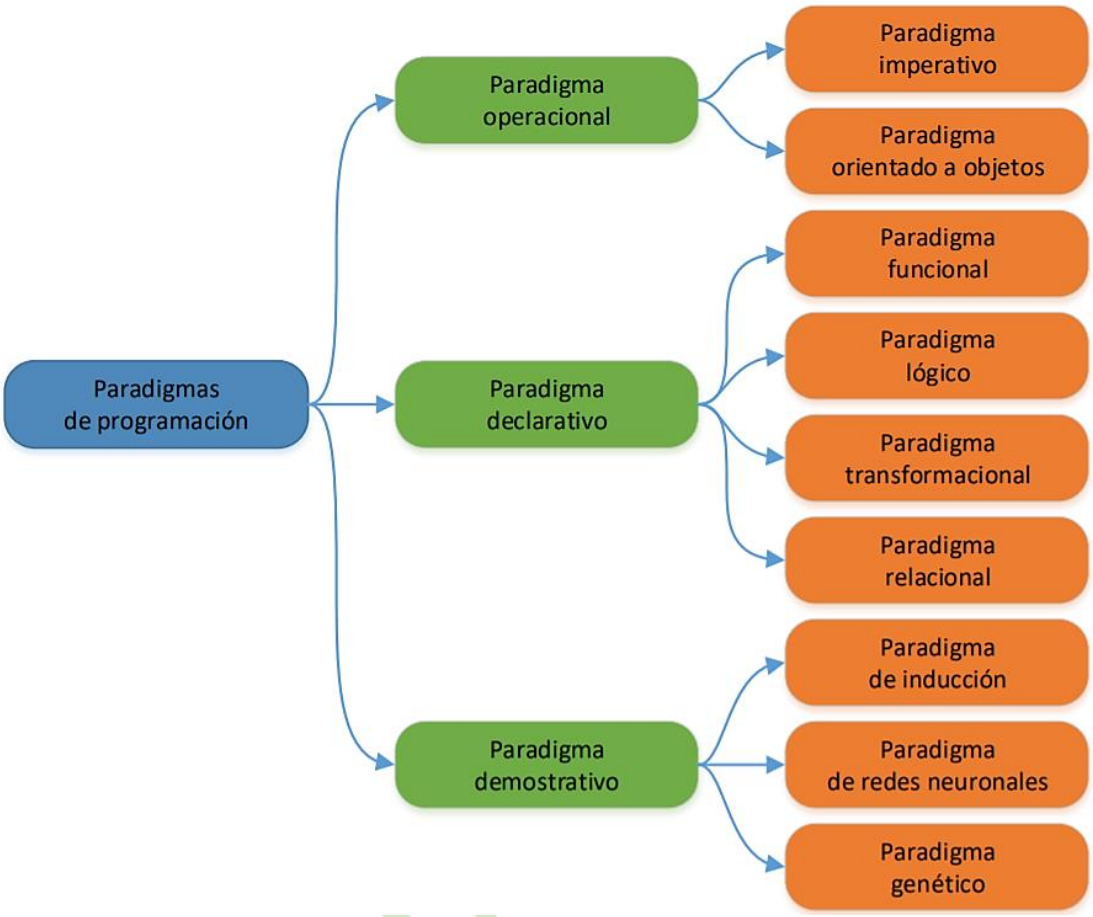
SOBRESCRITURA (*OVERRIDING*): Situación que ocurre cuando una clase hija le da una implementación más específica a un método implementado y heredado de una clase padre.

THREAD-SAFE: Múltiples hilos pueden acceder a ellos concurrentemente sin necesitar sincronización.

UNBOXING: Proceso de extraer un tipo básico de su correspondiente clase envoltorio.

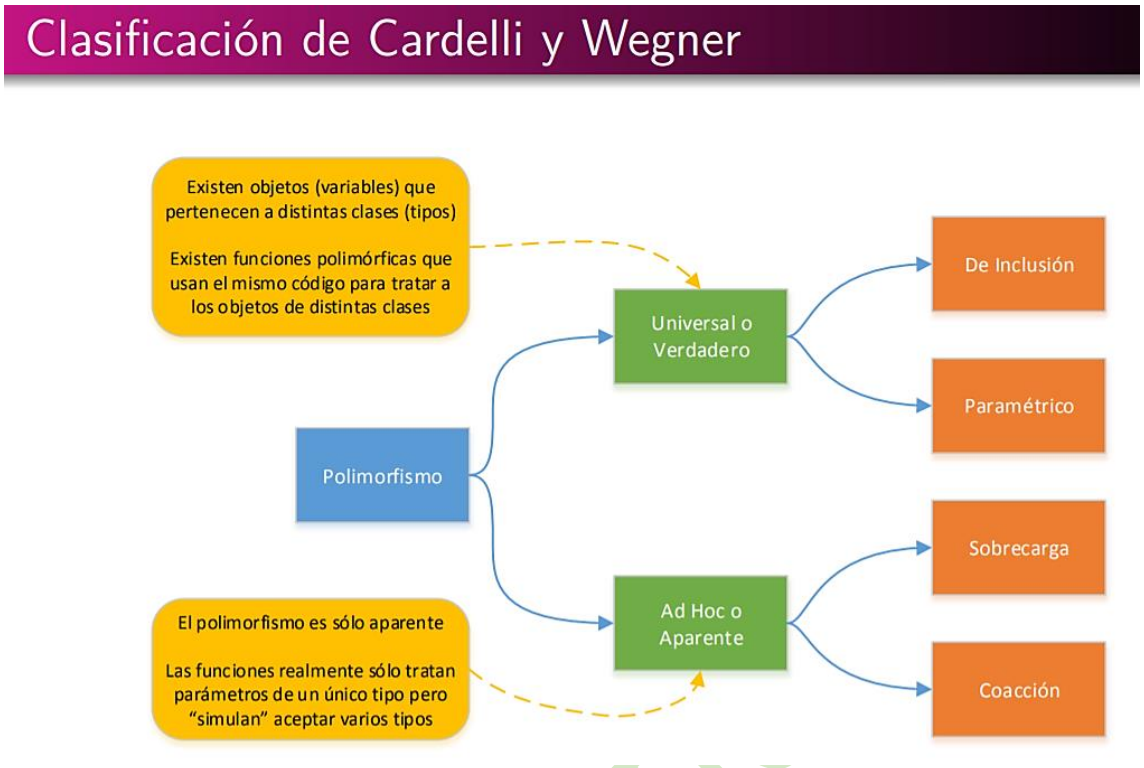
@informaticapau

PARADIGMAS DE PROGRAMACIÓN



@infor

POLIMORFISMO



@informati

OO Basics

Abstraction

Encapsulation

Polymorphism

Inheritance

OO Principles

Encapsulate what varies.

Favor composition over inheritence.

Program to interfaces, not implementations.

OO Patterns

Strategy – defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

@informatica